	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez.

Asignatura: Programación orientada a objetos.

Grupo: 07

No de Práctica(s): 8

Integrante(s): 323331229

426060019

323217156

312109239

323195386

321321976

No. de lista o brigada: 6

Semestre: 3ro

Fecha de entrega: 23/10/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **3**
1.1. Planteamiento del problema 3
1.2. Motivación 3
1.3. Objetivos 3

2. Marco Teórico **4**

3. Desarrollo **5**
3.1. Ejercicio 1 5
3.2. Ejercicio 2 6

4. Resultados **7**
4.1. Resultados del Ejercicio 1 7
4.2. Resultados del Ejercicio 2 8

5. Conclusiones **9**

1. Introducción

1.1. Planteamiento del problema

La práctica nos solicita utilizar nuestro LLM para investigar la construcción de la GUI para hacer login de usuario y contraseña, Además nos solicitan Calcular el impacto ambiental de tres entidades no relacionadas por herencia: Edificio, Auto y Bicicleta. Cada una de estas clases debe contener atributos y comportamientos exclusivos, pero unificarse implementando una interfaz común llamada HuellaCarbono con su respectivo método de cálculo getHuellaCarbono() el programa debe culminar en una clase principal que agrupe instancias de estos objetos en un ArrayList, recorriéndolos para mostrar los datos identificativos y la huella de carbono correspondiente a cada uno.

1.2. Motivación

Implementar el concepto de polimorfismo en un lenguaje de programación orientado a objetos.

1.3. Objetivos

- **General:** Implementar el principio de polimorfismo mediante la creación de interfaces y explorar la construcción de interfaces gráficas de usuario (GUI), para el desarrollo de un sistema que sea capaz de calcular y mostrar el impacto ambiental de entidades de diferente naturaleza.
- **Específicos:**
  - Con el apoyo de un modelo de lenguaje (LLM) investigar la estructura necesaria para construir una interfaz gráfica funcional de autenticación (inicio de sesión con usuario y contraseña).
  - Definir una interfaz común llamada HuellaCarbono con su método getHuellaCarbono() e implementarla en tres clases sin relación de herencia entre sí (Edificio, Auto y Bicicleta), asegurando que cada una conserve atributos y comportamientos exclusivos.
  - Agrupar varias instancias de las clases creadas en una sola estructura de datos (ArrayList de tipo HuellaCarbono), mostrando cómo objetos de distinta índole pueden ser tratados de forma homogénea mediante una interfaz común.

2. Marco Teórico

- **Polimorfismo:** Palabra de origen griego que significa "muchas formas". Se refiere a la propiedad por la que es posible enviar mensajes sintácticamente iguales a objetos de tipos distintos. Consiste en conseguir que un objeto de una clase se comporte como un objeto de cualquiera de sus subclases. Se puede aplicar tanto a métodos como a tipos de datos. Los métodos pueden evaluar y ser aplicados a diferentes tipos de datos de manera indistinta. Los tipos polimórficos son tipos de datos que contienen al menos un elemento, cuyo tipo no está especificado. [1]

- **Polimorfismo dinámico o paramétrico:** Es aquel en el que no se especifica el tipo de datos sobre el que se trabaja y, por ende, se puede recibir utilizar todo tipo de datos compatible. Este tipo de polimorfismo también se conoce como programación genérica. [1]

- **Polimorfismo estático (o ad hoc):** Es aquel en el que los tipos de datos que se pueden utilizar deben ser especificados de manera explícita antes de ser utilizados. [1]

- **instanceof:** Permite identificar el tipo de objeto enviado, es decir, no revisa la referencia sino el objeto en sí.[1]

- **Clases abstractas:** Una clase abstracta es una clase de la que no se pueden crear objetos, debido a que define la existencia de métodos, pero no su implementación, sus métodos son explícitos e implícitos. [1]

- **Interfaces:** Este tipo de diseños permiten establecer la forma de una clase (nombres de métodos, listas de argumentos y tipos de retorno), pero no bloques de código. Las interfaces pueden ser implementadas por cualquier clase. La clase que implementa una interfaz está obligada a definir (implementar) los métodos que la interfaz declaró. Todos los datos miembros (atributos) que se definen en una interfaz son públicos, estáticos y finales y dado que las interfaces no pueden instanciarse, resultan una buena herramienta para implantar grupos de constantes que pueden ser llamados sin crear objetos. [1]

3. Desarrollo

3.1. Ejercicio 1

Esta primera actividad consistió en emplear un modelo de lenguaje (LLM) como herramienta de investigación para determinar la estructura lógica y visual necesaria en la construcción de una interfaz gráfica de usuario (GUI),para un sistema de inicio de sesión con usuario y contraseña. Con esto el LLM nos dio la siguiente respuesta:

Para construir una interfaz gráfica (GUI) de inicio de sesión en Java, el estándar más utilizado para aplicaciones de escritorio tradicionales es la biblioteca Swing. La construcción se divide en los componentes visuales y la lógica de interacción.

- **Componentes Visuales Necesarios (Swing):** Estos son los elementos de la interfaz de usuario (UI) que el usuario verá y con los que interactuará:
- **JFrame:** La ventana principal que contiene toda la aplicación.
- **JPanel:** Un contenedor secundario que agrupa y organiza los elementos dentro de la ventana.
- **JLabel:** Etiquetas de texto no editables para indicar qué debe ingresar el usuario (ej. `"Usuario:" y "Contraseña:"`).
- **JTextField:** Una caja de texto de una sola línea donde el usuario escribe su nombre de usuario.
- **JPasswordField:** Una caja de texto especial que enmascara los caracteres (usualmente con asteriscos o puntos) para ocultar la contraseña.
- **JButton:** El botón que el usuario presiona para enviar los datos (ej. `"Iniciar Sesión"`).
- **JOptionPane:** (Opcional pero recomendado) Ventanas emergentes (pop-ups) para mostrar mensajes de error si las credenciales son incorrectas.
- **Estructura Lógica y Flujo:** La estructura debe seguir un flujo ordenado, separando el diseño visual de la lógica de autenticación (un principio básico del patrón Modelo-Vista-Controlador o MVC).
- **Configuración del Layout (Diseño):** Dentro del JFrame, se añade un JPanel al que se le asigna un "Layout Manager"(como GridLayout o GridBagLayout). Esto asegura que las etiquetas y los campos de texto estén alineados correctamente en una cuadrícula (ej. 2 columnas x 3 filas).
- **Asignación de Listeners (Manejadores de Eventos):** Al componente JButton se le debe añadir un ActionListener. Este es un bloque de código que se queda `"escuchando"` y se ejecuta únicamente cuando el usuario hace clic en el botón.

- **Extracción y Limpieza de Datos:** Dentro del método del ActionListener, el programa debe extraer los datos ingresados:

  Se lee el usuario con textField.getText().

  Se lee la contraseña usando passwordField.getPassword() (devuelve un arreglo de caracteres char[] por motivos de seguridad en memoria, no un String directo).

- **Capa de Autenticación (Lógica de Negocio):** Los datos extraídos no deben validarse directamente en el código de la interfaz gráfica. Deben enviarse a un método o clase separada que se encargue de comparar esas credenciales con una base de datos, un archivo o un sistema de validación.

- **Manejo de Resultados (Feedback al Usuario):** Autenticación Exitosa: Se destruye u oculta la ventana de login (usando frame.dispose()) y se instancia/muestra la ventana de la aplicación principal.

- **Autenticación Fallida:** Se limpia el JPasswordField y se muestra un mensaje de error (usando JOptionPane.showMessageDialog()) para que el usuario intente de nuevo.

3.2. Ejercicio 2

El segundo ejercicio de esta práctica se solicitó modelar y calcular el impacto ambiental de tres entidades que no guardan ninguna relación de herencia entre sí: `Edificio`, `Auto` y `Bicicleta`.

- Primero creamos la interfaz HuellaCarbono con su método getHuellaCarbono(), la cual implementaremos en las otras clases para relacionarlas sin tener que usar herencia.

- Después creamos las clases Auto, Bicicleta y Edificio con sus respectivos atributos exclusivos y sus getters y setters.

- Para el Auto, definimos marca, modelo, tipo y distanciaRecorrida, además de una constante FACTOR_EMISIÓN de 0.20.

- Para la Bicicleta, definimos atributos más simples: marca, modelo y tipo.

- Para el Edificio, definimos nombre, direccion, tipo, electricidad y consumoGas, junto con dos constantes: FACTOR_ELECTRICO (0.444) y FACTOR_GAS (2.1).

- Utilizando la interfaz, definimos el método getHuellaCarbono() para obtener el cálculo respectivo en cada clase: el Auto multiplica su distancia recorrida por su factor, el Edificio suma sus consumos de luz y gas multiplicados por sus respectivos factores, y la Bicicleta simplemente retorna 0.0 porque no contamina.

- Además utilizamos el método toString() en cada clase para poder imprimir la información básica (como la marca o el nombre) que identifica a cada objeto.

- En el main definimos una lista la cual va a recibir objetos tipo HuellaCarbono. Instanciamos un objeto de cada uno (auto, edificio, bici) pasándole los valores a sus atributos a través del constructor, y los guardamos en el arreglo.
- Finalmente recorremos esta lista con un ciclo for y por cada elemento en ella imprimimos su información general (invocando al objeto) y calculamos su huella (invocando a getHuellaCarbono()).

.

Como prueba instanciamos un objeto de cada clase pasándole sus datos iniciales específicos:

- Auto: Le pasamos la marca `"Toyota"`, el modelo `"Corolla"`, el tipo `"Sedán"` y una distancia recorrida de 100.0.
- Edificio: Lo instanciamos con el nombre `"Edificio X, Anexo FI"`, la dirección "Circuito Exterior, Ciudad Universitaria", el tipo `"Educacional"`, un consumo eléctrico de 200.0 y un consumo de gas de 50.0.
- Bicicleta: Le dimos la marca `"Trek"`, el modelo `"Tarmac"` y el tipo `"Montaña"`.

4. Resultados

4.1. Resultados del Ejercicio 1

Se muestran las respuestas obtenidas al consultar al LLM sobre como crear una GUI para hacer login de Usuario y Contraseña:

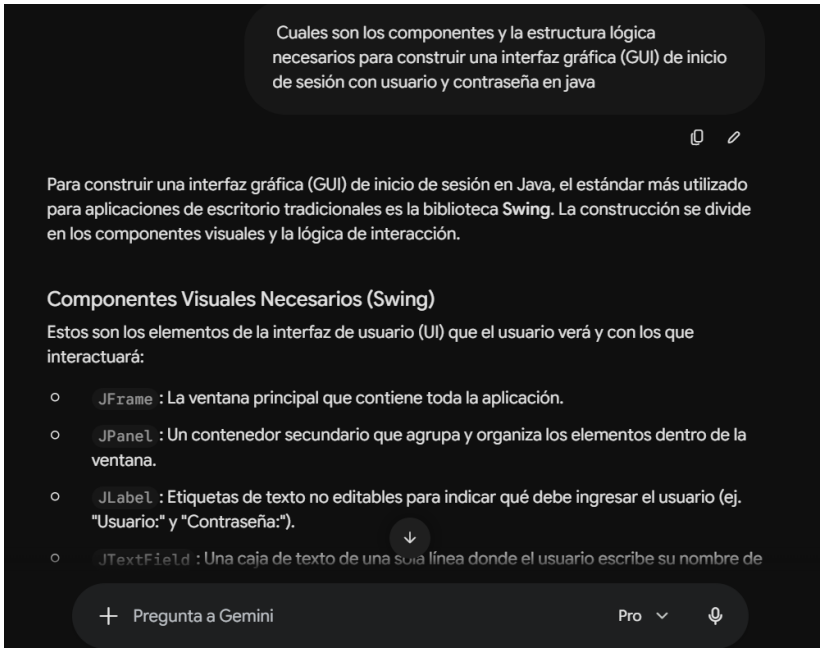


Figura 1: Respuesta proporcionada por Gemini.

4.2. Resultados del Ejercicio 2

El diagrama UML que modela la jerarquía.

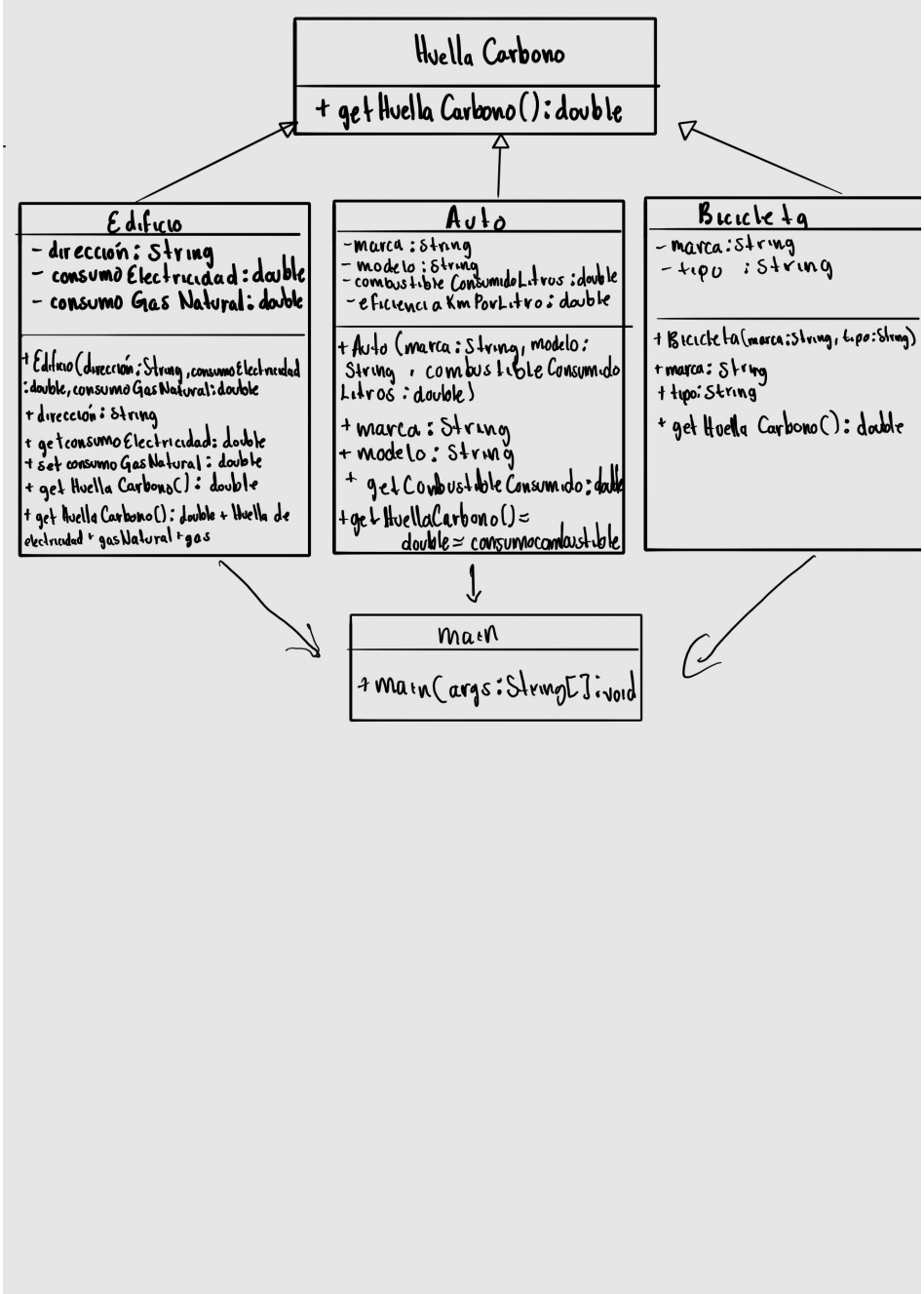


Figura 2: Diagrama UML representando la jerarquía y estructura.

Se realizó la prueba de ejecución del programa principal (`Main.java`) en la terminal, donde se instanciaron los tres objetos de prueba y se agregaron a una lista de tipo `HuellaCarbono`.

```
PS C:\Users\o-ale\OneDrive\Documentos\POO> javac -d test mx\unam\fi\die\poo\g7\src\Práctica_8\Ejercicio_2\*.java
PS C:\Users\o-ale\OneDrive\Documentos\POO> java -cp test mx.unam.fi.die.poo.g7.src.Práctica_8.Ejercicio_2.Main
Auto [marca: Toyota, modelo: Corolla, tipo: Sedán]
20.0
Edificio [nombre: Edificio X, Anexo FI, direccion: Circuito Exterior, Ciudad Universitaria, tipo: Educacional]
193.8
Bicicleta [marca: Trek, modelo: Tarmac, tipo: Montaña]
0.0
PS C:\Users\o-ale\OneDrive\Documentos\POO>
```

Figura 3: Ejecución del programa.

5. Conclusiones

El análisis de los resultados obtenidos demuestra la aplicación práctica del concepto de polimorfismo a través del uso de interfaces en Java. Al definir e implementar la interfaz `HuellaCarbono`, se logró un comportamiento polimórfico en tres entidades que carecen de una jerarquía de herencia directa (`Auto`, `Edificio` y `Bicicleta`). Esto permitió abstraer su comportamiento, agrupando objetos de distintos tipos bajo una misma estructura de datos (un `ArrayList` de tipo `HuellaCarbono`) y enviándoles un mensaje uniforme mediante el método `getHuellaCarbono()`. La ejecución correcta de fórmulas independientes para cada entidad confirma que el programa resolvió de forma dinámica la implementación específica correspondiente en tiempo de ejecución, evidenciando cómo el polimorfismo facilita el diseño de sistemas escalables.

Por otro lado, la investigación estructurada sobre la construcción de interfaces gráficas (GUI) mediante Swing permitió analizar la arquitectura subyacente de las aplicaciones de escritorio. El diseño de una pantalla de inicio de sesión comprobó teóricamente la necesidad de aplicar el patrón Modelo-Vista-Controlador (MVC), destacando que el desacoplamiento entre los componentes visuales (como `JFrame` y `JPasswordField`) y la lógica de negocio (validación de credenciales a través de *listeners*) es fundamental para mantener la integridad y cohesión del software orientado a objetos.

Referencias

[1] Olguín, J. A. S. G. R. A. S. (s.f.-c). Polimorfismo. Unidades de Apoyo Para el Aprendizaje - CUAED - UNAM. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3062/mod_resource/content/1/UAPA-Polimorfismo/index.html